3D Cardiac MRI Autoencoder Architectures

Technical Report — Architecture Review

AI Agent Experiment | June 2026

All models trained at latent_dim=120 | 100 train / 20 val / 30 test patients | ED+ES frames

Introduction

This report describes the 3D autoencoder architectures developed for cardiac MRI representation learning. The input data consists of 3D volumes of shape (1, 32, 128, 128) — one channel, 32x128x128 spatial resolution. All models compress this volume into a latent vector of dimension 120.

The report covers both the original architectures (developed manually) and the new architectures proposed and implemented by an AI agent. For each architecture, the following aspects are described:

- Theoretical description — the mathematical and conceptual design
- Code implementation — key classes and blocks
- Motivation — why it could improve upon the reference model
- Results — validation R^2, test R^2, convergence, and analysis

The two original architectures AE3dLinear and AE3dFCDeep_VAE are described without results, as they were not included in the AI agent experiment batch.

Summary Comparison Table

All models evaluated at latent_dim = 120, baseline hyperparameters (unless noted). Highlighted row = best performing model.

Model	Val R^2 mean	Test R^2 mean	Train R^2 mean	Val MSE mean	Best Epoch
AE3dFCDeep (baseline)	0.688	0.750	0.879	220.2	76
AE3dCurrent	0.774	0.810	0.879	165.3	38
AE3dConv	0.743	0.777	0.860	177.5	45
AE3dLinear	—	—	—	—	—
AE3dFCDeep_VAE	—	—	—	—	—
AE3dAttention (AI)	0.742	0.774	0.846	191.6	63
AE3dDilated (AI)	0.762	0.789	0.883	169.2	43
AE3dDilatedAttention (AI)	**0.804**	**0.822**	**0.878**	**151.2**	**58**
AE3dSeparableDilated (AI)	0.786	0.809	0.868	155.8	37

PART I — Original Architectures

AE3dCurrent — Original Model

Theoretical Description

AE3dCurrent is a standard 3D convolutional autoencoder with a linear bottleneck. The encoder applies three successive downsampling convolutional blocks, reducing the spatial dimensions by a factor of 8 in each axis. A non-downsampling bottleneck convolution then increases the channel depth, and a fully-connected layer maps the flattened feature map to the latent vector z.

Encoder path: *(1,32,128,128) → [Conv×2, MaxPool] → (8,16,64,64) → [Conv×2, MaxPool] → (16,8,32,32) → [Conv×2, MaxPool] → (32,4,16,16) → [Conv×2] → (64,4,16,16) → Flatten(65536) → Linear →* $z \in \mathbb{R}$

The decoder mirrors this path: a linear layer expands z back to 65,536 features, which are reshaped and passed through three transposed convolution blocks to reconstruct the original volume. The final activation is a Sigmoid. The loss is the mean squared error (MSE) between input and reconstruction.

Code Implementation

Class: AutoEncoder3D_Current. Uses Conv3DBlock (two Conv3d → InstanceNorm3d → ReLU, then MaxPool3d) for the encoder, and UpConv3DBlock (ConvTranspose3d → Conv3d → InstanceNorm3d → ReLU × 2) for the decoder.

- enc1–enc3: Conv3DBlock with downsample=True (3 blocks)
- bottleneck_conv: Conv3DBlock with downsample=False
- fc_enc / fc_dec: Linear(65536, latent_dim) and inverse
- dec1–dec3: UpConv3DBlock
- Flattened feature size: 64 × 4 × 16 × 16 = 65,536

Motivation vs. Baseline

This is the baseline reference architecture, predating the AI agent experiment. It serves as the primary point of comparison for all other models. Its design is intentionally simple: a symmetric encoder-decoder with standard convolutions and a single fully-connected bottleneck.

Results (latent_dim = 120, lr = 5e-5, best_epoch = 38)

Metric	Value
Validation R^2 (mean ± std)	0.774 ± 0.118
Validation R^2 (median)	0.816
Test R^2 (mean ± std)	**0.810 ± 0.080**
Train R^2 (mean)	0.879
Validation MSE (mean)	165.3
Validation RMSE (mean)	12.6
Best epoch	38

AE3dCurrent converges quickly (best epoch 38), suggesting the architecture is relatively easy to optimise. Despite its simpler design compared to AE3dFCDeep, it achieves a substantially better validation R^2 (0.774 vs 0.688). This counterintuitive result — where the shallower model outperforms the deeper one — is the central motivation for the AI agent exploration. The lower train R^2 gap (0.879 vs 0.688 val) indicates some generalisation difficulty is present, but less than in AE3dFCDeep. The bottleneck at (64, 4, 16, 16) preserves more spatial structure before compression, which may explain the advantage.

AE3dFCDeep — Deep Fully-Connected Bottleneck

Theoretical Description

AE3dFCDeep extends the original model by adding a fourth downsampling block and deepening the bottleneck. The encoder compresses the volume more aggressively: after four downsampling steps, a two-layer bottleneck convolution at 128 channels is followed by a strided convolution that reduces the feature map to (128, 1, 4, 4). This 2,048-dimensional vector is then projected to the latent space via a linear layer.

Encoder path: *(1,32,128,128) → enc1–enc4 (4× MaxPool) → (64,2,8,8) → [Conv×2@128] → (128,2,8,8) → Conv(stride=2) → (128,1,4,4) → Flatten(2048) → Linear →* $z \in \mathbb{R}$

The bottleneck is much smaller than in AE3dCurrent (2,048 vs 65,536 pre-linear features), meaning more compression happens in convolutional space before the linear projection. This is the AI agent's reference model.

Code Implementation

Class: AutoEncoder3D_FCDeep. Four encoder blocks, followed by a sequential bottleneck and a strided convolution for final compression.

- enc1–enc4: Conv3DBlock with downsample=True (4 blocks)
- bottleneck_conv: Sequential [Conv3d(64,128) → IN → ReLU] × 2
- final_down: Conv3d(128,128, kernel=2, stride=2) → (128,1,4,4)
- fc_enc / fc_dec: Linear(2048, latent_dim) and inverse
- Flattened feature size: 128 × 1 × 4 × 4 = 2,048

Motivation vs. AE3dCurrent

The deeper architecture was designed to capture more abstract features through additional non-linear processing. The smaller pre-linear feature map (2,048 vs 65,536) forces more information to be encoded in convolutional representations, which could lead to better generalisation by reducing the burden on the linear bottleneck. In practice, however, this model underperforms AE3dCurrent in this experiment.

Results (latent_dim = 120, lr = 1e-5, best_epoch = 76)

Metric	Value
Validation R^2 (mean ± std)	0.688 ± 0.205
Validation R^2 (median)	0.744
Test R^2 (mean ± std)	0.750 ± 0.124
Train R^2 (mean)	0.879
Validation MSE (mean)	220.2
Validation RMSE (mean)	14.4
Best epoch	76 (slowest convergence)

AE3dFCDeep is the weakest model in this batch, despite — or because of — its greater depth. It also converges the slowest (best epoch 76) and has the highest R^2 standard deviation on validation (0.205), indicating inconsistent per-patient reconstruction quality. The large train-val gap (train R^2 0.879, val R^2 0.688) suggests overfitting. Notably, it was trained at a lower learning rate (1e-5 vs 5e-5 for AE3dCurrent), which may compound the problem. This model serves as the reference baseline for the AI agent, which used it as a starting point for architectural modifications.

NB Arthur → AE3dFCDeep was the best performing model on average across all latent dimensions, including 120 latent dims. This training may just have been unlucky, and hasn't been retrained to show the variability of the performance of the AE models trained.

AE3dConv — Fully Convolutional Bottleneck

Theoretical Description

AE3dConv removes all linear layers from the bottleneck. Instead of flattening to a vector, the encoder reduces the feature map to a small 3D tensor of shape (C, 1, 2, 2), where C = latent_dim / 4. This tensor is then flattened to produce the latent vector z. The bottleneck therefore remains in convolutional space throughout.

Encoder path: *(1,32,128,128) → enc1–enc4 → (64,2,8,8) → Conv@128 → (128,2,8,8) → Conv(stride=2) → (128,1,4,4) → Conv(stride=2) → (C,1,2,2) → Flatten → z* $\in \mathbb{R}$*^(4C)*

The constraint latent_dim = 4C means the latent dimension must be a multiple of 4. The latent space retains a mild spatial structure, as the channel-spatial factorisation is preserved until the final flatten.

Code Implementation

Class: AutoEncoder3D_Conv. No nn.Linear in encoder or decoder; the bottleneck is purely convolutional.

- enc1–enc4: Conv3DBlock with downsample=True
- pre_latent: Conv3d(64,128, k=3, p=1) → IN → ReLU
- reduce_to_1x4x4: Conv3d(128,128, k=2, stride=2) — spatiotemporal stride
- reduce_to_latent: Conv3d(128, C, k=(1,2,2), stride=(1,2,2)) → (C,1,2,2)
- Decoder mirrors with ConvTranspose3d steps; no Linear layers
- latent_channels C = latent_dim // 4 = 30 at latent_dim=120

Motivation vs. AE3dFCDeep

The fully convolutional design avoids the information bottleneck introduced by flattening to a 1D vector through a linear layer. By keeping the latent representation as a structured tensor, the model may preserve local spatial relationships that are discarded in AE3dFCDeep's linear projection. This is inspired by fully convolutional network designs where spatial context is considered valuable even at the bottleneck.

Results (latent_dim = 120, lr = 5e-5, best_epoch = 45)

Metric	Value
Validation R^2 (mean ± std)	0.743 ± 0.168
Validation R^2 (median)	0.798
Test R^2 (mean ± std)	0.777 ± 0.168
Train R^2 (mean)	0.860
Validation MSE (mean)	177.5
Validation RMSE (mean)	13.1
Best epoch	45

AE3dConv sits between AE3dFCDeep and AE3dCurrent in performance, improving on the former (+0.055 val R^2 mean) but falling short of the latter (-0.031). The high R^2 standard deviation on the test set (0.168) suggests the fully convolutional bottleneck produces inconsistent results across patients. The lower train R^2 (0.860 vs 0.879 for AE3dFCDeep) may indicate underfitting: without a linear projection, the model may lack the capacity to fully compress 120 dimensions. The convergence speed is moderate (epoch 45).

AE3dLinear — Linear Autoencoder (PCA Equivalent)

Theoretical Description

AE3dLinear is a purely linear, non-convolutional autoencoder. The encoder flattens the entire input volume and applies a single linear transformation to produce z. The decoder applies the inverse linear transformation and reshapes. No non-linearities are used at any stage.

Encoder: $x \in \mathbb{R}^{524288} \rightarrow Linear(524288, d) \rightarrow z \in \mathbb{R}^{d}$

Decoder: $z \in \mathbb{R}^{d} \rightarrow Linear(d, 524288) \rightarrow reshape \rightarrow \hat{x} \in \mathbb{R}^{(1,32,128,128)}$

Under MSE loss and without non-linearities, this model is theoretically equivalent to PCA: the learned encoder-decoder pair converges to a projection onto the top-d principal components of the data (up to rotation within the subspace). It therefore serves as a principled linear baseline.

Code Implementation

Class: AutoEncoder3D_Linear. Minimal implementation.

- flatten: nn.Flatten()
- fc_enc: Linear(524288, latent_dim, bias=True)
- fc_dec: Linear(latent_dim, 524288, bias=True)
- No activations, no normalisation, no convolutions
- Input size: 1 × 32 × 128 × 128 = 524,288

Motivation

AE3dLinear provides the theoretical lower bound for reconstruction quality under a given latent dimension. Any non-linear model that fails to exceed PCA performance is learning nothing beyond what linear dimensionality reduction already captures. It is therefore an essential diagnostic tool rather than a competitive architecture.

Results

AE3dLinear was not included in the AI agent experiment batch. Results from prior experiments (see project weekly summaries) indicate that PCA consistently outperforms all convolutional autoencoders tested so far, particularly at higher latent dimensions. This remains a key open challenge for the project.

AE3dFCDeep_VAE — Variational Autoencoder

Theoretical Description

AE3dFCDeep_VAE adapts the AE3dFCDeep architecture into a Variational Autoencoder (VAE). The convolutional encoder-decoder backbone is identical to AE3dFCDeep; only the bottleneck changes. Instead of a single deterministic linear projection, the encoder outputs two vectors μ and log σ² of dimension d, which parameterise a Gaussian distribution over the latent space.

Bottleneck: $h \in \mathbb{R}^{2048} \rightarrow fc\_mu \rightarrow \mu \in \mathbb{R}^{d}$ and $h \rightarrow fc\_logvar \rightarrow \log \sigma^2 \in \mathbb{R}^{d}$

Reparameterisation: $z = \mu + \varepsilon \cdot \sigma,\ \varepsilon \sim N(0, I)$ *[training only; z = μ at inference]*

Loss: $L = MSE(x, \hat{x}) + \beta \cdot KL(N(\mu, \sigma^2) \,||\, N(0, I))$

The β parameter (with linear warm-up over beta_warmup_epochs) controls the trade-off between reconstruction quality and latent space regularity. At β = 0 the model reduces to a standard AE; as β increases, the latent space becomes more structured and continuous.

Code Implementation

Class: AutoEncoder3D_FCDeep_VAE. Identical to AE3dFCDeep except at the bottleneck.

- fc_enc replaced by: fc_mu = Linear(2048, d) and fc_logvar = Linear(2048, d)
- reparameterize(mu, logvar): sampling active during training, z = mu at eval
- forward() returns (x_recon, z, mu, logvar) — 4 outputs vs 2 for standard AE
- Training loss adds β·KL term, computed externally in the training loop
- VAE-specific hyperparameters tuned by Optuna: β = 0.00022, beta_warmup_epochs = 41

Motivation vs. AE3dFCDeep

The VAE introduces a structured, continuous latent space. This is not primarily aimed at better reconstruction — the KL term may slightly degrade MSE — but at producing a latent space where interpolation, sampling, and downstream statistical modelling are more meaningful. This is particularly relevant for the project's long-term goal of latent motion modelling of cardiac dynamics.

Results

AE3dFCDeep_VAE was not included in the AI agent experiment batch. It is currently being tuned with Optuna. Prior results with Optuna hyperparameters on the standard AE3dFCDeep show R^2 improvement from 0.75 to 0.80; the VAE equivalent is still under evaluation.

PART II — AI Agent Architectures

All AI agent architectures share the same backbone as AE3dFCDeep (4 encoder blocks, deep bottleneck, 2048-dimensional pre-linear feature map, identical decoder). Only the encoder blocks are modified. This design choice isolates the effect of the new building blocks while keeping the comparison fair.

Three new building blocks were introduced:

- **SEBlock3D** — Squeeze-and-Excitation channel attention
- **DilatedConv3DBlock** — Dilated 3D convolutions with progressive dilation (1, 2, 4, 1)
- **SeparableConv3DBlock** — Depthwise separable 3D convolutions with dilation

AE3dAttention — SE Channel Attention

Theoretical Description

AE3dAttention replaces all Conv3DBlocks in the encoder with AttentionConv3DBlocks, which append a Squeeze-and-Excitation (SE) module after each standard convolutional block.

SE mechanism: $z = GlobalAvgPool(x) \rightarrow Linear(C, C/r) \rightarrow ReLU \rightarrow Linear(C/r, C) \rightarrow Sigmoid \rightarrow scale \times x$

The SE block computes a per-channel weight vector by globally pooling the spatial dimensions (squeeze), passing through a small bottleneck MLP (excitation), and multiplying back onto the feature map (recalibration). The reduction ratio r = 16. The result is that the network learns to suppress uninformative channels and amplify relevant ones at each scale.

Code Implementation

Classes: SEBlock3D, AttentionConv3DBlock, AutoEncoder3D_Attention.

- SEBlock3D: AdaptiveAvgPool3d(1) → view → Linear(C, C//16) → ReLU → Linear(C//16, C) → Sigmoid → expand × x
- AttentionConv3DBlock: Conv3DBlock(in, out, downsample) → SEBlock3D(out)
- enc1–enc4 in AutoEncoder3D_Attention: all AttentionConv3DBlock
- Bottleneck, linear layers, and decoder: identical to AE3dFCDeep
- Extra parameters vs AE3dFCDeep: small (SE MLPs are C/16 bottlenecks)

Motivation vs. AE3dFCDeep

In cardiac MRI, different feature channels encode distinct anatomical structures (myocardium, blood pool, background). SE attention allows the model to dynamically weight these channels depending on the input, potentially focusing compression on the most diagnostically relevant features. The overhead is minimal.

Results (latent_dim = 120, lr = 1e-5, best_epoch = 63)

Metric	Value
Validation R^2 (mean ± std)	0.742 ± 0.119
Validation R^2 (median)	0.781
Test R^2 (mean ± std)	0.774 ± 0.098
Train R^2 (mean)	0.846
Validation MSE (mean)	191.6
Validation RMSE (mean)	13.6
Best epoch	63

AE3dAttention underperforms AE3dCurrent and even AE3dConv (val R^2 0.742 vs 0.774 and 0.743). The SE blocks alone do not improve reconstruction quality at this latent dimension. This suggests that channel recalibration is not the limiting factor in AE3dFCDeep — the issue lies rather in the receptive field or the capacity of individual convolutions. The slow convergence (best epoch 63) and lower train R^2 (0.846) indicate the SE mechanism may add optimisation difficulty without sufficient benefit at this scale.

AE3dDilated — Dilated Convolutions

Theoretical Description

AE3dDilated replaces the standard Conv3DBlocks with DilatedConv3DBlocks, using a progressive dilation schedule across encoder layers: dilation = 1, 2, 4, 1. Dilated convolutions insert zeros between kernel elements, effectively increasing the receptive field without increasing the number of parameters or reducing resolution.

Effective receptive field with dilation d, kernel k: $k\_eff = k + (k-1)(d-1)$ → $k=3, d=2: k\_eff=5,\ d=4: k\_eff=9$

The final encoder block returns to dilation=1 to stabilise feature maps before the bottleneck. The dilation progression (1→2→4→1) is a common pattern borrowed from semantic segmentation (DeepLab, WaveNet) to combine local and global context.

Code Implementation

Classes: DilatedConv3DBlock, AutoEncoder3D_Dilated.

- DilatedConv3DBlock: Conv3d(k=3, padding=dilation, dilation=dilation) × 2 → IN → ReLU → MaxPool
- enc1: DilatedConv3DBlock(1, 8, dilation=1)
- enc2: DilatedConv3DBlock(8, 16, dilation=2)
- enc3: DilatedConv3DBlock(16, 32, dilation=4)
- enc4: DilatedConv3DBlock(32, 64, dilation=1) ← stabilisation
- Bottleneck, linear layers, and decoder: identical to AE3dFCDeep

Motivation vs. AE3dFCDeep

Standard 3×3×3 convolutions have a limited receptive field per layer. In 3D cardiac MRI, relevant structures (e.g., full ventricular contour, wall thickness gradients) span relatively large spatial extents. Dilated convolutions allow the encoder to capture these long-range dependencies without additional downsampling, preserving resolution while expanding context.

Results (latent_dim = 120, lr = 5e-5, best_epoch = 43)

Metric	Value
Validation R^2 (mean ± std)	0.762 ± 0.143
Validation R^2 (median)	0.806
Test R^2 (mean ± std)	0.789 ± 0.127
Train R^2 (mean)	0.883
Validation MSE (mean)	169.2
Validation RMSE (mean)	12.7
Best epoch	43

AE3dDilated is a clear improvement over AE3dFCDeep (+0.074 val R^2 mean) and approaches AE3dCurrent (-0.012). The train R^2 is the highest in the entire batch (0.883), indicating strong expressive capacity. The train-val gap remains, suggesting some overfitting, but the validation performance is substantially better than the baseline. Fast convergence (epoch 43) is a positive sign. This result confirms that increasing the receptive field via dilation is a key driver of improvement in this architecture family.

AE3dDilatedAttention — Dilated Convolutions + SE Attention [BEST]

Theoretical Description

AE3dDilatedAttention combines dilated convolutions with Squeeze-and-Excitation attention in each encoder block. Each DilatedAttentionConv3DBlock applies a DilatedConv3DBlock followed by an SEBlock3D. The dilation schedule is identical to AE3dDilated (1, 2, 4, 1).

Block: $x \rightarrow$ *DilatedConv3DBlock(dilation=d)* $\rightarrow h \rightarrow$ *SEBlock3D* $\rightarrow y = h \cdot \sigma(MLP(GAP(h)))$

The combination is synergistic: dilated convolutions capture multi-scale spatial context across the volume, and the SE module then selects which channels carry the most relevant information at each scale. Together they address two complementary limitations of standard convolutions: restricted spatial context and uniform channel weighting.

Code Implementation

Classes: DilatedAttentionConv3DBlock, AutoEncoder3D_DilatedAttention.

- DilatedAttentionConv3DBlock: DilatedConv3DBlock(in, out, dilation, downsample) → SEBlock3D(out)
- enc1: DilatedAttentionConv3DBlock(1, 8, dilation=1)
- enc2: DilatedAttentionConv3DBlock(8, 16, dilation=2)
- enc3: DilatedAttentionConv3DBlock(16, 32, dilation=4)
- enc4: DilatedAttentionConv3DBlock(32, 64, dilation=1)
- Bottleneck, linear layers, and decoder: identical to AE3dFCDeep

Motivation vs. AE3dFCDeep

The hypothesis is that dilation and SE attention address different, complementary weaknesses of the base model: dilation improves the spatial context captured at each layer, while SE reweights the resulting channels to emphasise the most informative ones. This should yield better representations than either mechanism alone — as confirmed by the results.

Results (latent_dim = 120, lr = 5e-5, best_epoch = 58)

Metric	Value
Validation R² (mean ± std)	**0.804 ± 0.073**
Validation R² (median)	**0.824**
Test R² (mean ± std)	**0.822 ± 0.069**
Train R² (mean)	0.878
Validation MSE (mean)	**151.2**
Validation MAE (mean)	4.54 ← lowest in batch
Validation RMSE (mean)	**12.0**
Best epoch	58

AE3dDilatedAttention is the best performing model in this batch across all metrics. Val R² mean (0.804) exceeds AE3dCurrent by +0.030 and AE3dFCDeep by +0.116. Test R² (0.822) is the highest. Notably, the R² standard deviation is the lowest in the batch (val: 0.073, test: 0.069), indicating more consistent reconstruction quality across patients — a clinically important property. The train-val gap is small (0.878 vs 0.804), suggesting good generalisation. The val MAE (4.54) is also the lowest, confirming superior pixel-level accuracy. Convergence is moderate (epoch 58), balanced between speed and quality.

This model should be prioritised for Optuna hyperparameter tuning and multi-latent-dim evaluation.

AE3dSeparableDilated — Depthwise Separable Dilated Convolutions

Theoretical Description

AE3dSeparableDilated replaces standard dilated convolutions with depthwise separable dilated convolutions. Each convolution is factorised into a depthwise step (one filter per input channel, applied independently) and a pointwise step (1×1×1 convolution mixing channels).

Standard Conv3d: $C\_in \times C\_out \times k^3$ *parameters*

Separable Conv3d: $C\_in \times k^3$ *(depthwise)* $+ C\_in \times C\_out$ *(pointwise)* $\approx k^3$ *times fewer parameters*

For k=3 in 3D, this reduces parameters by approximately 27×. The dilation schedule (1, 2, 4, 1) is maintained. The bottleneck convolutions also use SeparableConv3DBlocks.

Code Implementation

Classes: SeparableConv3DBlock, AutoEncoder3D_SeparableDilated.

- SeparableConv3DBlock: [depthwise Conv3d(groups=C_in) → pointwise Conv3d(1×1×1) → IN → ReLU] × 2 → MaxPool
- enc1–enc4: SeparableConv3DBlock with dilation = 1, 2, 4, 1
- bottleneck_conv: Sequential of two SeparableConv3DBlocks (dilation=1)
- final_down, linear layers, and decoder: identical to AE3dFCDeep
- Note: decoder uses standard UpConv3DBlocks (not separable)

Motivation vs. AE3dFCDeep

Depthwise separable convolutions reduce the parameter count dramatically, which can act as an implicit regulariser and reduce overfitting on small datasets. Combined with dilation for large receptive fields, the model attempts to achieve the spatial context benefits of AE3dDilated with fewer parameters, potentially improving generalisation on the limited training set (~100 patients).

Results (latent_dim = 120, lr = 5e-5, best_epoch = 37)

Metric	Value
Validation R^2 (mean ± std)	0.786 ± 0.093
Validation R^2 (median)	0.814
Test R^2 (mean ± std)	0.809 ± 0.072
Train R^2 (mean)	0.868
Validation MSE (mean)	155.8
Validation RMSE (mean)	12.3
Best epoch	37 (fastest convergence)

AE3dSeparableDilated achieves strong performance — second best in the batch (val R^2 0.786, test R^2 0.809) — with the fastest convergence of all models (epoch 37). The R^2 standard deviation is low (val: 0.093, test: 0.072), close to AE3dDilatedAttention, confirming that the separable + dilated combination produces consistent reconstructions. The lower train R^2 (0.868 vs 0.878 for AE3dDilatedAttention) and slightly lower validation metrics suggest that the parameter reduction, while beneficial for generalisation, may impose a capacity limit.

This model is an attractive alternative to AE3dDilatedAttention when computational efficiency matters: it trains faster, uses fewer parameters, and achieves only marginally lower reconstruction quality. It warrants further evaluation with Optuna tuning.

Discussion and Recommendations

Key Findings

- **AE3dDilatedAttention is the best model** (val R^2 0.804, test R^2 0.822), with the lowest variance across patients — a combination of dilated convolutions and SE attention.
- **Dilated convolutions are the primary driver** of improvement: AE3dDilated alone (+0.074 vs AE3dFCDeep) outperforms SE attention alone (+0.054). The combination is synergistic (+0.116).
- **AE3dSeparableDilated is a strong efficiency trade-off**: slightly below AE3dDilatedAttention but fastest to train and fewest parameters.
- **SE attention alone (AE3dAttention) does not help**: it even underperforms AE3dConv, suggesting that channel reweighting is only beneficial when combined with better spatial context.
- **The baseline AE3dFCDeep remains the weakest model**, despite its depth. This highlights that aggressive compression in convolutional space (to 2048 features before the linear layer) is not inherently beneficial.
- **All results are at a single operating point** (latent_dim=120, one split, baseline hyperparameters for new models). Conclusions should be validated across multiple latent dimensions and with Optuna tuning.

Recommended Next Steps

- Run AE3dDilatedAttention and AE3dSeparableDilated with Optuna hyperparameter tuning (lr, weight_decay, patience)
- Evaluate both across the full latent_dim range (1 to 240) to assess behaviour at low compression
- Compare best AI agent model against PCA on the test set
- Consider applying SE attention or dilation to the decoder as well — the current asymmetry (new encoder, standard decoder) may limit gains
- Investigate the high R^2 variance in AE3dConv and AE3dAttention — patient-specific failure modes may be informative